Week 11 (Days 71-77): Convolutional networks

Status: written in full (Phase 3). Titles match `SYLLABUS.md`.

Week goals

- Compute a convolution by hand, with loops and as an im2col matrix multiply, and get the output-size arithmetic right every time.
- Know the architectural ideas from LeNet to ResNet, and show in a run why residual connections help depth.
- Train a CNN on CIFAR-10-small on a CPU and compare it fairly with an MLP.
- Use a pretrained network: frozen features vs fine-tuning.
- Debug deep nets with a fixed checklist (initial loss, overfit one batch, sanity checks).
- Turn audio into images (spectrograms) and reuse what you know about CNNs.
- Ship a classifier with an error analysis (weekly project).

```
flowchart LR
  A[Day 71 conv from scratch] --> B[Day 72 architectures]
  B --> C[Day 73 CIFAR CNN]
  C --> D[Day 74 transfer learning]
  D --> E[Day 75 debugging]
  E --> F[Day 76 audio pipelines]
  F --> G[Day 77 project + quiz]
```

Compute note (honest). All runs are CPU-only (8 threads, no GPU). We therefore use **CIFAR-10-small**, 20,000 32x32 images, and short training (5-10 epochs). The accuracies below are far below what the same architectures reach on full CIFAR-10 with a GPU and long schedules. They are good for *comparisons inside this course*, not for comparing with papers.

Datasets: CIFAR-10-small from OpenML, dataset 40926 (`https://www.openml.org/d/40926`), loaded by `load_cifar_small()` in `code/common.py` and split once in `code/week-11/_cifar.py` into 15,000 train / 2,500 validation / 2,500 test (seed 0), normalised with training statistics. (The original Toronto tarball was unreachable from this box when the course was built; see `data/README.md`.) Fashion-MNIST via torchvision as in Week 10. ImageNet ResNet-18 weights from `https://download.pytorch.org/models/resnet18-f37072fd.pth` (about 45 MB). Day 76 generates synthetic audio, so nothing is downloaded.

Day 71: Convolutions from scratch (and as im2col matmul)

Review today: Day 70 (+1), Day 68 (+3), Day 64 (+7), Day 50 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. An MLP on images treats pixel (3, 4) and pixel (3, 5) as unrelated inputs, and must learn "edge detector" again at every position. A convolution slides one small filter across the image: the same weights everywhere (*weight sharing*), looking only at a neighbourhood (*local connectivity*). That builds in translation equivariance: shift the input, and the feature map shifts.

Definition. For input x with C channels and a filter w of shape (C, k, k), output at (i, j) = b + Sum_c Sum_u Sum_v w[c, u, v] . x[c, i.s + u, j.s + v]. Deep-learning libraries compute this cross-correlation (no kernel flip) and call it convolution. A layer has K filters, so its weight is (K, C, k, k) and it outputs K channels.

Output size. With input size n, kernel k, padding p, stride s: **out = ?(n + 2p - k)/s? + 1**. Memorise it. "Same" padding for odd k and stride 1 is p = (k-1)/2.

Parameters. K.C.k.k + K. This does not depend on the image size, unlike a dense layer.

Receptive field. How many input pixels can influence one output unit. Stacking 3x3 convolutions grows it by 2 per layer; pooling or stride multiplies the growth.

im2col. Copy every kxkxC patch into a column of a big matrix (C.k.k rows, one column per output position). Then convolution is one matrix multiply: W (K x C.k.k) @ cols. Fast libraries use this or similar tricks (Winograd, FFT, direct kernels), because matmul is what CPUs and GPUs are best at.

Pooling. Max or average over a window, usually 2x2 stride 2: halves the resolution, adds a little translation invariance, no parameters.

Block 2: Worked example (45 min)

Run `code/week-11/day71_conv_scratch.py`. **Verified:** - A 1-D-like edge detector [-1, 1] on a step signal gives **[0, 3, 3]**: it responds where intensity changes. - Loop convolution vs `torch.nn.functional.conv2d`: max error **1.1e-14**; im2col version: **8.9e-15**. Both are correct to round-off. - Timing for an 8x3x32x32 batch and 16x3x3x3 filters, across my reruns: loops **208-244 ms**, im2col **2.1-3.1 ms**, torch **0.27-0.39 ms** (one rerun on a heavily loaded box gave torch 15.28 ms, so always time twice). im2col is roughly 80-100x faster than loops; the library roughly another 8-9x. - Output sizes for the script's six cases: **30, 32, 32, 16, 24, 112** (check each with the formula in Block 4). - `Conv2d(3, 16, 3)` has **448** parameters. A dense layer mapping the same 3x32x32 input to the same 16x30x30 output would need **44,251,200**. - The stack in the script has a receptive field of **16** pixels.

Block 3: Coding lab (2h)

1. (40 min) Write `conv2d_loops(x, w, b, stride, pad)` for a single image. Test against torch on random data with stride 2 and padding 1.
2. (40 min) Write `im2col` and `conv2d_im2col`. Time both against torch for an 8x3x32x32 batch and 16 filters (then try 3x64x64 yourself).
3. (20 min) Implement 2x2 max pooling with `reshape` and `max` (no loops).
4. (20 min) Hand-design a 3x3 vertical edge filter and apply it to a Fashion-MNIST image. Plot the output.

Block 4: Practice problems (1h)

1. n = 32, k = 3, p = 0, s = 1. *Answer: 30.*
2. n = 32, k = 3, p = 1, s = 1. *Answer: 32.*
3. n = 32, k = 5, p = 2, s = 2. *Answer: ?(32+4-5)/2?+1 = 16.*
4. n = 224, k = 7, p = 3, s = 2. *Answer: 112.*
5. Parameters of Conv2d(64, 128, 3)? *Answer: 64.128.9 + 128 = 73,856.*
6. Receptive field of three stacked 3x3 convs (stride 1)? *Answer: 7.*

Block 5: Review (1h)

Without notes: write the output-size formula, the parameter-count formula, and explain im2col in three sentences. Self-check: - [ ] I can reproduce the 448 vs 44,251,200 comparison. - [ ] I can explain why the library is faster than im2col. - [ ] I know that frameworks compute cross-correlation.

Going deeper (optional)

Read the "Convolution arithmetic" guide by Dumoulin and Visin (linked in `docs/resources.md`), especially transposed convolutions, which you will need for upsampling and generative models.

Interview-style question (15 min, written)

"Why do CNNs need far fewer parameters than MLPs on images, and what assumption do they make?" Weight sharing and locality; assumption: useful features are local and position-independent.

Common mistakes / how to debug

- Off-by-one output sizes: always check with a dummy tensor, `model(torch.zeros(1, 3, 32, 32)).shape`.
- Channels-last (NHWC) arrays passed to PyTorch, which expects NCHW. See Day 75 for why `view` is wrong here.
- Forgetting padding, so feature maps shrink each layer and the Flatten size is wrong.

What to do next

Day 72 stacks convolutions into the famous architectures, and tests the key idea behind ResNet.

Day 72: CNN architectures: LeNet to ResNet ideas

Review today: Day 71 (+1), Day 69 (+3), Day 65 (+7), Day 51 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

A short history in ideas (not dates to memorise). - **LeNet-style:** conv -> pool -> conv -> pool -> dense. Small, for digits. - **AlexNet-style:** the same pattern, much bigger, with ReLU, dropout and GPUs. Showed deep CNNs win on ImageNet. - **VGG-style:** only 3x3 convolutions, stacked in blocks, doubling channels when halving resolution. Two 3x3 layers see 5x5 with fewer parameters than one 5x5 and an extra non-linearity. - **Inception-style:** parallel branches with different kernel sizes; 1x1 convolutions to reduce channels cheaply. - **ResNet:** add the input of a block to its output, y = F(x) + x. If the best thing a block can do is nothing, it only has to learn F = 0. Gradients flow through the identity path unchanged, so very deep networks train. When the shape changes, the skip path uses a 1x1 conv with stride. - **Global average pooling:** average each final feature map to one number instead of flattening. Far fewer parameters in the head, and the network accepts any input size.

The degradation problem. Before ResNet, adding layers to an already deep plain network made *training* error worse, not just test error. That is an optimisation problem, not overfitting.

Block 2: Worked example (45 min)

Run `code/week-11/day72_architectures.py`. **Verified:** - It prints the layer-by-layer shapes of a LeNet-style net (**61,706** parameters), a small VGG-style net (**307,498**) and one residual block (**4,672**). - Plain vs residual on Fashion-MNIST (5,000 training images, lr 0.02, 5 epochs), test accuracy:

depth	plain	residual
2 blocks	0.5750	0.5480
10 blocks	0.6120	**0.7145**

At depth 2 the skip connection made no positive difference (slightly worse, within what one seed can tell). At depth 10, residual wins by 0.10. Gradient norms at the first layer at initialisation: plain **0.172**, residual **0.428**: more signal reaches the bottom of the residual net. Accuracies are low because training is very short; the comparison is what matters.

Block 3: Coding lab (2h)

1. (40 min) Write a `ResidualBlock(cin, cout, stride)` with the 1x1 projection skip. Test shapes for (16->16, stride 1) and (16->32, stride 2).
2. (40 min) Repeat the plain-vs-residual experiment at depth 20. Record first-layer gradient norms too.
3. (20 min) Replace Flatten + Linear with global average pooling in the VGG-style net. Count parameters before and after.
4. (20 min) Draw your network as a diagram with shapes on each arrow.

Block 4: Practice problems (1h)

1. Two 3x3 convs (C->C) vs one 5x5 conv (C->C), ignoring biases. Parameters? *Answer: 18C^2 vs 25C^2.*
2. Why does a 1x1 conv with stride 2 appear in ResNet's skip path? *Answer: to match channels and resolution so x can be added to F(x).*
3. Is "deeper plain net has higher *training* error" overfitting? *Answer: no; it is an optimisation failure.*
4. Global average pooling on a 128x4x4 feature map produces? *Answer: a 128-vector.*

Block 5: Review (1h)

Write one sentence per architecture idea. Self-check: - [ ] I can write a residual block from memory. - [ ] I can explain the depth-10 result and why depth 2 showed nothing.

Extra notes: reading an architecture (study before the interview question)

When you meet a new architecture, read it in this order. First, the **stem**: how the image enters (a 7x7 stride-2 conv plus pooling in ImageNet ResNets, a single 3x3 conv for 32x32 inputs). The stem decides how fast resolution falls, and for small images an ImageNet stem throws away most of the pixels in the first two layers. Second, the **stages**: groups of blocks at one resolution.

Count how many times resolution halves and channels double; most CNNs keep the compute per stage roughly constant this way, because halving H and W divides the cost by 4 and doubling channels multiplies it by 4 (C_in x C_out doubles twice). Third, the **block**: plain conv-BN-ReLU, residual basic block, bottleneck (1x1 reduce, 3x3, 1x1 expand) or inverted bottleneck (MobileNetV2). Fourth, the **head**: global average pooling and a linear layer, or flatten and dense layers. Finally, count parameters and multiply-adds per stage; that tells you where memory and time go.

Apply this to the three Day-72 nets. LeNet-style: two conv-pool stages on 28x28, then three dense layers; most of its 61,706 parameters sit in the dense layers, not the convolutions. VGG-style: 3x3 convolutions only, channels 32->64->128, and its 307,498 parameters are dominated by the last convolutions and the head. The residual block: two bias-free 3x3 convs of 16 channels (2 x 16.16.9 = 4,608; the bias is dropped because BN's shift does the same job) plus two BN layers with a scale and shift per channel (2 x 2 x 16 = 64), total 4,672; verify by printing `named_parameters()`. Doing this count yourself is a reliable way to understand a design and to catch bugs where a layer is not what you think.

Going deeper (optional)

Read the ResNet paper (He et al., 2015) section on the degradation problem; also look at depthwise-separable convolutions (MobileNet) for efficient models on devices.

Interview-style question (15 min, written)

"Why do residual connections help train deep networks?" Identity path for gradients, easy to learn identity, smoother optimisation; mention BN's role too.

Common mistakes / how to debug

- Adding the skip *after* the final ReLU of the block in a way that changes the design without noticing.
- Shape mismatch on the addition: print shapes of F(x) and skip(x).
- Concluding from depth-2 experiments that residuals don't matter.

What to do next

Day 73 trains a CNN on CIFAR-10-small and compares it with an MLP at fair settings.

Day 73: Training a CNN on CIFAR-10 (CPU-sized)

Review today: Day 72 (+1), Day 70 (+3), Day 66 (+7), Day 52 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The data. Ten classes of small colour photos (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). Objects vary in position, scale, pose and background. This is where the CNN's assumptions pay off.

A fair comparison. Same split, same epochs, same optimiser family, LR checked for each model. Report parameters and time per epoch, not just accuracy.

Augmentation for natural images. Random horizontal flips (a car facing left is still a car) and random crops after padding by 4 pixels (small translations). Never flip vertically for these classes.

Reading train vs validation. A large gap means overfitting; if both are low, the model or the training is too weak.

Block 2: Worked example (45 min)

Run `code/week-11/day73_cifar_cnn.py` (about 3.5 minutes here). 10 epochs, SGD lr 0.01 with momentum for every model. **Verified:**

model	params	train acc	val acc	time/epoch
MLP	1,578,506	0.7628	0.5020	0.2 s
CNN	307,498	0.5912	0.5516	4.5 s
CNN + BN	308,394	0.9372	**0.7444**	8.1 s

model	params	train acc	val acc	time/epoch
CNN + BN + flip/crop aug	308,394	0.7928	0.7268	7.0 s

Notes: - The MLP has 5x the parameters of the CNN and overfits hard (0.76 train, 0.50 val). - BN made the CNN much better in 10 epochs. While building this lesson, BN at lr 0.05 was tried first and failed badly (val 0.22 after one epoch vs 0.48 at lr 0.01), so all models use lr 0.01. - Augmentation reduced the train/val gap from 0.19 to 0.07 but did not raise validation accuracy in 10 epochs. Augmentation usually needs longer training to pay off. The Day-86 capstone trains longer. - Per-class validation accuracy for CNN + BN: cat **0.538**, dog **0.53** and bird **0.581** are the hardest; automobile **0.878** is among the easiest.

Block 3: Coding lab (2h)

1. (30 min) Build your own CNN: two conv blocks (conv-BN-ReLU-conv-BN-ReLU-pool), then global average pooling and a linear layer.
2. (45 min) Train 10 epochs with and without augmentation; log to CSV.
3. (25 min) Compute the per-class accuracy and the confusion matrix. Which pairs are confused?
4. (20 min) Plot a training curve (train loss, val acc per epoch).

Block 4: Practice problems (1h)

1. Why is a vertical flip a bad augmentation here? *Answer: upside-down cars and ships don't occur in test data; it teaches invariances that are false.*
2. The MLP's 0.26 train/val gap vs the CNN+BN 0.19 gap: what does each say? *Answer: both overfit; the MLP generalises far worse despite (because of) more parameters.*
3. Why report time per epoch? *Answer: cost matters; a better model at 40x the time may not be the right choice.*

Block 5: Review (1h)

Explain the four-row table to a colleague in five sentences. Self-check: - [ ] I can say why augmentation didn't help accuracy in 10 epochs. - [ ] I know which classes were hardest and can guess why.

Extra notes: what a fair CPU experiment looks like

Write down the protocol before you look at results. For this lesson: one fixed split (seed 0), normalisation statistics from the training images only, the same number of epochs (10), the same batch size, the same optimiser family (SGD with momentum) and one LR that was checked to be stable for every model (0.01). Record the parameter count, time per epoch, final training accuracy and validation accuracy, and never touch the 2,500 test images until a model is chosen. That protocol is why the table can be trusted for *comparison*, even though every number in it is far below published CIFAR-10 results.

Notice the time column. The MLP is about 20-40 times faster per epoch than the CNNs on this CPU, because a convolution does many multiply-adds per parameter (the same 3x3 filter is applied at every position) while a dense layer does one per parameter. Parameter count therefore says little about compute. For a convolution layer, multiply-adds = K . C . k . k . H_out . W_out. For the first layer of our CNN (32 filters, 3 channels, 3x3, 32x32 output) that is 32 . 3 . 9 . 1,024 = 884,736 per image; you can check how that compares with the dense layer of the MLP (3,072 x 512 = 1,572,864) and then see why the deeper conv layers dominate. On a GPU these differences shrink because convolutions parallelise well, so always time on the hardware you will deploy on.

Finally, BN changed the result most. BN normalises each channel with batch statistics during training and running averages during evaluation, so forgetting `model.eval()` changes test results; the Day-75 checklist tests exactly this.

Going deeper (optional)

Try `torch.optim.lr_scheduler.OneCycleLR` with 20 epochs (about 3 minutes per run). The capstone does this.

Interview-style question (15 min, written)

"You have a fixed compute budget of 10 epochs. Do you add augmentation?" Discuss the measured trade-off and how you would decide on validation.

Common mistakes / how to debug

- Normalising with per-image statistics in training and dataset statistics at test.

- Applying augmentation at evaluation.
- Comparing an MLP at its best LR with a CNN at a bad LR.

What to do next

Day 74 starts from a network that has already seen a million images.

Day 74: Transfer learning and fine-tuning a pretrained CNN

Review today: Day 73 (+1), Day 71 (+3), Day 67 (+7), Day 53 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. The early layers of a network trained on ImageNet detect edges, textures and parts that are useful for almost any photo. Reusing them means you need far less data and compute for a new task.

Three options, from cheapest to most flexible. 1. **Frozen features:** run the pretrained network up to the penultimate layer once, save the features, train a linear classifier (logistic regression) on top. 2. **Fine-tune the top:** replace the final layer, unfreeze the last stage, train with a small LR. 3. **Fine-tune everything:** usually with a smaller LR for early layers (discriminative LRs).

Practical details. - Use the **same preprocessing** the weights were trained with (resize, ImageNet mean/std). torchvision's weight enums expose `transforms()` and metadata. - Resize small images up (here 32->64; ImageNet models expect larger inputs; 64 is a CPU compromise). - BN layers in a frozen backbone should stay in eval mode. - Pretrained weights carry the biases and licence terms of their training data. Read the model card.

Block 2: Worked example (45 min)

Run `code/week-11/day74_transfer.py` (about 1 minute plus a 45 MB weight download the first time). Only **2,000** training images, validated on 2,500, inputs 64x64. **Verified:**

approach	val acc	time
ResNet-18 from scratch, 6 epochs	0.3852	about 21-30 s
frozen ImageNet features (512-d) + logistic regression	0.6424	13 s incl. feature extraction
fine-tune layer4 + fc, 4 epochs	**0.6856**	8 s

Fine-tuning trained **8,398,858** of **11,181,642** parameters. torchvision's metadata reports **69.758** ImageNet top-1 accuracy for these weights. With 2,000 images, the pretrained features beat training from scratch by about 0.26-0.30, and cost less time. Compare: the Day-73 CNN + BN needed 15,000 images to reach 0.7444.

Block 3: Coding lab (2h)

1. (40 min) Reproduce the frozen-features approach with your own feature extraction loop (`model.fc = nn.Identity()`).
2. (40 min) Fine-tune with two LR groups (backbone 1e-4, head 1e-2). Compare with the script.
3. (40 min) Vary the training set size: 500, 2,000, 8,000. Plot val accuracy for "from scratch" vs "frozen features". Where do they cross, if at all?

Block 4: Practice problems (1h)

1. Why use ImageNet mean/std rather than CIFAR statistics for the pretrained model? *Answer: the weights expect inputs distributed as in their training.*
2. When is frozen features + linear model the right choice? *Answer: little data, little compute, need for a quick strong baseline.*
3. What fraction of parameters were trainable in our fine-tune? *Answer: 8,398,858 / 11,181,642 ~= 75% (layer4 is most of ResNet-18).*

Block 5: Review (1h)

Self-check: - [ ] I can list three transfer strategies and when to use each. - [ ] I know why preprocessing must match. - [ ] I can freeze and unfreeze parameters with `requires_grad_`.

Extra notes: choosing between the three options

Think of two axes: how much labelled data you have, and how similar your images are to ImageNet photos. Little data and similar images (our 2,000 CIFAR photos): frozen features and a linear model are a strong, cheap baseline; fine-tuning the top stage adds a little (0.6424 -> 0.6856 here). Lots of data and similar images: fine-tune everything with a small LR. Little data and very different images (medical scans, spectrograms, satellite bands): frozen high-level features fit less well; fine-tune earlier layers and use strong augmentation, or look for a backbone pretrained on similar data. Lots of data and very different images: training from scratch becomes competitive, but pretrained initialisation still usually speeds up convergence.

Two engineering details matter in practice. First, cache the frozen features: extracting 512-d features once and training logistic regression on them takes seconds, while running the backbone every epoch wastes most of the compute. Second, check what your pretrained model saw. ImageNet contains classes that overlap with CIFAR-10 (many dog and bird species, cars, ships), which is part of why transfer works so well here; it also means a "held-out" evaluation can be optimistic if your test images resemble the pretraining data very closely. For a real product, read the model card and the licence of the weights, and keep a record of the exact weight file (here `resnet18-f37072fd.pth`) so results can be reproduced.

Going deeper (optional)

Look at the torchvision "Models and pre-trained weights" page (linked in `docs/resources.md`) and compare ResNet-18 with a MobileNet on the same 2,000 images.

Interview-style question (15 min, written)

"You have 300 labelled images of product defects. What do you build first?" Pretrained backbone, frozen features baseline, strong augmentation, careful validation (maybe cross-validation), then fine-tuning.

Common mistakes / how to debug

- Forgetting to replace the 1000-class head.
- Frozen layers still updating because the optimiser received all parameters and `requires_grad` was not turned off.
- Resizing inconsistently between training and evaluation.

What to do next

Day 75 builds the habits that tell you, fast, whether a training run is broken.

Day 75: Debugging deep nets: overfit one batch, loss curves, sanity checks

Review today: Day 74 (+1), Day 72 (+3), Day 68 (+7), Day 54 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. Neural-network bugs rarely crash. They train, a bit worse. So you need checks that give a clear yes/no.

The checklist, in order. 1. **Initial loss.** For C balanced classes and a sensible init, cross-entropy at step 0 ~= ln C (2.303 for 10). Far above that means bad init or scale; far below means a leak. 2. **Overfit one small batch.** Train on 32 examples only; the loss should go almost to zero. If it doesn't, the model, loss or optimiser is broken. Nothing else is worth trying until this passes. 3. **Shuffled-label sanity check.** Train on randomly permuted labels: test accuracy must drop to chance. If it doesn't, test data is leaking. 4. **Look at the data after the pipeline.** Plot a batch exactly as the model sees it (after augmentation and normalisation) with labels. 5. **Loss curves.** Train and val on the same plot. Diverging -> overfit; flat -> LR or bug; spiky -> LR too high or bad batches. 6. **Gradient norms per layer.** Zero -> dead path or detached tensor; huge -> explosion.

Classic bugs (each measured below): softmax before `cross_entropy` (double softmax), missing `zero_grad`, wrong reshape (`view` vs `permute`), labels misaligned with inputs, forgetting `eval()`.

Block 2: Worked example (45 min)

Run `code/week-11/day75_debugging.py` (about 8 s). **Verified:** 1. Initial loss **2.2930** vs ln 10 = **2.3026**. 2. Overfitting one batch of 32: loss **2.2777 -> 0.0127 -> 0.0021 -> 0.0007** at steps 1, 50, 100, 200. 3. Same model and budget with planted bugs:

run	train acc	test acc
correct	0.8794	0.8381
labels shuffled (sanity check)	0.1200	0.0889
softmax before cross_entropy	0.6586	0.6497
forgot zero_grad	0.1036	0.1000

The double-softmax bug still "works", just much worse: exactly the silent kind of bug. Forgetting `zero_grad` destroyed training completely here. 4. Converting NHWC to NCHW: `permute(0, 3, 1, 2)` and `view(N, C, H, W)` give different tensors. The first channel's first values were [0, 3, 6, 9] with permute (correct: every third value is the same channel) vs [0, 1, 2, 3] with view (mixes channels). 5. Gradient norms per weight: **0.0802, 0.3391, 0.8398** from first to last layer: healthy, decreasing towards the input but not vanishing.

Block 3: Coding lab (2h)

1. (30 min) Write `debug_checks(model, loader)` that prints initial loss vs ln C and runs the one-batch overfit test.
2. (45 min) Plant each of these bugs in your Day-73 CNN and record the symptom: labels shifted by one example, `model.eval()` never called back to `train()`, normalisation applied twice, augmentation on the test set.
3. (45 min) Write a one-page "symptom -> likely cause" table from your results.

Block 4: Practice problems (1h)

1. Initial loss is 9.8 on a 10-class problem. First suspect? *Answer: initial logits too large (init scale), or labels/loss mismatch.*
2. One-batch overfit stalls at loss 1.5. What next? *Answer: check that the optimiser has the model's parameters, LR, that gradients are non-zero, that inputs vary, and that no layer detaches the graph.*
3. Shuffled-label test accuracy is 0.45. What does it mean? *Answer: leakage between train and test, or a bug in the shuffle.*
4. Why is `view` wrong for NHWC->NCHW? *Answer: it reinterprets memory without moving data; channels are interleaved.*

Block 5: Review (1h)

Memorise the six-step checklist. Self-check: - [ ] I can predict the initial loss for any C. - [ ] I have run the one-batch overfit test on my own model.

Extra notes: reading the bug table

Why did "forgot zero_grad" destroy training completely while double softmax only hurt? Without `zero_grad`, PyTorch adds each new gradient to the old ones, so after n steps the update is the sum of n gradients: an effective learning rate that grows every step until the weights blow up and predictions become constant (0.1000 test accuracy is exactly one class out of ten). Double softmax is milder: applying softmax to the outputs and then `cross_entropy` (which applies log-softmax again) squeezes the inputs into [0, 1], so the gradients become very small and training is slow, but it still moves in the right direction (0.6497). Recognising the *shape* of a failure-blow-up, plateau at chance, slow but steady-narrows the list of suspects quickly.

Going deeper (optional)

Read Andrej Karpathy's blog post "A Recipe for Training Neural Networks" (linked in `docs/resources.md`); compare its steps with ours.

Interview-style question (15 min, written)

"Your model trains but is 5 points worse than a colleague's identical-looking code. How do you find the difference?" Diff configs and data pipelines, compare initial loss, compare a batch tensor by tensor, fix seeds and compare the first few steps' losses.

Common mistakes / how to debug

- Skipping the one-batch test because "the model is standard".
- Debugging on the full dataset (slow feedback). Use a small subset.
- Changing several things at once while debugging.

What to do next

Day 76 applies the same tools to audio: turn sound into a picture and use a CNN.

Day 76: Audio/image pipelines for multimedia devs (spectrograms, augmentation)

Review today: Day 75 (+1), Day 73 (+3), Day 69 (+7), Day 55 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. If you have worked with media codecs or DSP, you already know the short-time Fourier transform (STFT): cut the signal into overlapping windows and take the FFT of each. The magnitude over time x frequency is a **spectrogram**, an image. CNNs are good at images, so many audio classifiers are CNNs on (log-)spectrograms.

Parameters you choose. - **Sample rate** fs: here 8,000 Hz (1-second clips = 8,000 samples). - **n_fft** (window length): 256 -> frequency resolution fs/n_fft = **31.25 Hz** per bin, and n_fft/2 + 1 = **129** bins. - **hop length**: 128 samples = **16 ms** between frames. - **log magnitude**: loudness perception is roughly logarithmic and the log compresses the dynamic range so the CNN sees structure, not a few huge values. Mel scaling (not used here) warps frequency to match hearing.

Audio augmentation. Time shift, added noise, gain, and **SpecAugment**-style masking (zero a random band of frequencies or a random span of time frames).

Image pipelines. `torchvision.transforms.v2` composes resize, crop, flip, colour jitter and normalisation and applies them consistently to images (and boxes/masks if needed).

Honesty note. No audio dataset is downloaded. The script **generates** four classes of 1-second clips (steady tone, up-chirp, beeps, tone + harmonic) with heavy noise (std 2.0), so the labels are known exactly. It teaches the pipeline, not real-world audio performance.

Block 2: Worked example (45 min)

Run `code/week-11/day76_audio_image_pipelines.py` (about 11 s). **Verified:** - 1,600 waveforms of 8,000 samples -> spectrograms of shape **(1600, 129, 63)**: 129 frequency bins, 63 frames. - It saves `out/day76_spectrograms.png` with one example per class. - Small CNN on spectrograms, 400 held-out clips: **0.8775**; with SpecAugment **0.8850**. - Baseline, logistic regression on the time-averaged spectrum: **0.7500**; per class tone **0.58**, chirp **0.88**, beeps **0.81**, harmonic **0.73**. Averaging over time throws away exactly what distinguishes a tone from its pulsed (beeps) and swept (chirp) versions, so the time-frequency CNN wins. - A `transforms.v2` pipeline on a CIFAR image outputs shape **(3, 32, 32)**.

When the noise was lower while building the lesson, every model reached 1.0 accuracy, which teaches nothing; the noise was raised to make the comparison informative.

Block 3: Coding lab (2h)

1. (40 min) Compute a spectrogram with `torch.stft` yourself; verify the shape for n_fft 512 and hop 256.
2. (40 min) Add time-shift and gain augmentation to the waveform before the STFT. Does accuracy change?
3. (40 min) Optional real audio: record a few seconds of your own voice saying two different words (your own data, no download), compute spectrograms and look at them. Do not train on 4 clips; just look.

Block 4: Practice problems (1h)

1. fs = 16,000, n_fft = 512. Bin width and number of bins? *Answer: 31.25 Hz, 257 bins.*
2. Hop 160 at 16 kHz is how many ms? *Answer: 10 ms.*
3. Why log-magnitude? *Answer: compresses dynamic range; closer to perception; more stable inputs.*
4. Why did the time-averaged baseline confuse tones and beeps? *Answer: their average spectra are similar; the difference is in time.*

Block 5: Review (1h)

Self-check: - [ ] I can choose n_fft and hop for a target time and frequency resolution. - [ ] I can explain SpecAugment. - [ ] I can state honestly what a synthetic benchmark can and cannot show.

Extra notes: designing the time-frequency trade-off

The STFT cannot be sharp in both time and frequency at once. A longer window (larger n_fft) gives finer frequency bins but blurs events in time; a shorter one does the opposite. Speech systems often use 25 ms windows with a 10 ms hop at 16 kHz (n_fft 400, hop 160), which is short enough to follow phonemes and long enough to resolve the harmonics of a voice. Our clips use 32 ms windows (256 samples at 8 kHz) and a 16 ms hop, giving 63 frames per second of audio. Work out these numbers for any pipeline you build and write them in the config, because a model trained on one setting will see shifted, stretched "images" under another.

The window function matters too. Cutting a sine wave abruptly makes its energy leak into neighbouring bins. A Hann window tapers the ends, trading slightly wider peaks for much less leakage; `torch.stft(..., window=torch.hann_window(n_fft))` applies it. Because the window tapers, consecutive frames must overlap (here by 50%) so that no part of the signal is ignored.

For image pipelines the same discipline applies: decide the resize policy (shorter side, centre crop, or letterbox), the colour space and the normalisation, and apply the same deterministic transforms at evaluation. Random augmentations belong only in training. The `transforms.v2` API makes this explicit by building two separate pipelines, one with random ops for training and one without for validation and test.

Going deeper (optional)

Look at `torchaudio.transforms.MelSpectrogram` and the SpecAugment paper (Park et al., 2019, linked in `docs/resources.md`).

Interview-style question (15 min, written)

"How would you build a keyword-spotting model for a device?" Log-mel features, small CNN, augmentation (noise, shift, SpecAugment), quantisation, latency budget, false-accept vs false-reject trade-off.

Common mistakes / how to debug

- Mixing sample rates between training and inference.
- Forgetting the window function, causing spectral leakage.
- Normalising each clip separately at training and globally at inference.

What to do next

Day 77 is the weekly project: a classifier with honest model selection and error analysis.

Day 77: PROJECT W11: image or audio classifier; Quiz 11

Review today: Day 76 (+1), Day 74 (+3), Day 70 (+7), Day 56 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson: the project brief (1h15)

Brief. Build an image classifier for Fashion-MNIST (or use the Day-76 audio pipeline if you prefer, with the same deliverables). Compare a linear baseline and at least three CNN variants, choose on validation, test once, and analyse the errors: 1. Per-class accuracy. 2. The most-confused class pairs (both directions together) and their share of all errors. 3. Confidence on correct vs wrong predictions, and the **expected calibration error (ECE)**: bin predictions by confidence, and average |accuracy - confidence| weighted by bin size. 4. A grid of the most confident errors, with true/predicted labels, and a written interpretation.

Why error analysis? Andrew Ng's advice: look at the errors before choosing what to work on. If 58% of errors are between tops and shirts, a better optimiser will not fix it; better data or features for those classes might.

Block 2: Worked example: the reference project (45 min)

Run `code/week-11/day77_week_project.py` (about 1.5 minutes). Split: 20,000 train, 5,000 val (from the 60,000 training images), full 10,000 test. CNNs: two conv blocks with BN and dropout, SGD momentum lr 0.02, cosine schedule, 6 epochs. **Verified:**

model	train acc (10k)	val loss	val acc	time
logistic regression (200 iterations, did not fully converge)	-	-	0.8290	20 s
cnn16	0.9112	0.2840	**0.9000**	6 s
cnn32	0.9082	0.2930	0.8940	22 s
cnn32 + flip	0.9012	0.2986	0.8916	32 s

cnn16 is selected on validation (the wider nets did not help in 6 epochs). **Test accuracy 0.8910**, 1,090 errors.

Error analysis: - Per-class test accuracy: Shirt **0.624** is by far the worst; Pullover 0.854, Coat 0.855, T-shirt/top 0.869; Bag 0.972, Sandal 0.968, Trouser 0.967 are the best. - Top confused pairs: T-shirt/top-Shirt **203**, Coat-Shirt **153**, Pullover-Coat **139**, Pullover-Shirt **132**. These four pairs hold **58%** of all errors: all upper-body garments. - Mean confidence: correct **0.916**, wrong **0.637**. Still, **104** errors had confidence above 0.9. - ECE **0.0127**: the model is well calibrated on average. - `out/day77_confident_errors.png` shows the 12 most confident errors. Look at them: several are hard even for a person at 28x28.

Block 3: Coding lab: your project (2h)

1. (20 min) Baseline and split.
2. (50 min) Three or more variants; log them.
3. (20 min) Select on validation; test once.
4. (30 min) Error analysis and a one-page write-up ending with "what I would do next, and why, based on the errors".

Rubric (self-grade out of 20)

criterion	points
Baseline + >=3 variants, logged	4
Validation-only selection, single test evaluation	4
Per-class accuracy and confused pairs with share of errors	4
Calibration (confidence gap and ECE) computed correctly	4
Error images inspected and a next step justified by them	4

Block 4: Practice problems (1h)

1. Why combine both directions of a confused pair? *Answer: the confusion is symmetric in cause (similar classes); one number per pair ranks problems better.*
2. A model has ECE 0.0127 but 104 errors with confidence > 0.9. Contradiction? *Answer: no; ECE is an average over bins and the high-confidence bin is mostly correct.*
3. What would you try next for Shirt? *Answer: look at examples; consider higher resolution, more data or augmentation for that class, or accept the ambiguity in the labels.*

Block 5: Review (1h)

Take the weekly quiz below, closed book.

Going deeper (optional)

Implement temperature scaling (fit one scalar T on validation to minimise NLL) and recompute ECE.

Interview-style question (15 min, written)

"Your classifier is 89% accurate. Your manager asks how to get to 95%. What do you do first?" Error analysis: categorise the errors, estimate the ceiling for each category, then prioritise.

Common mistakes / how to debug

- Computing ECE with predicted-class probability vs true-class probability inconsistently.
- Reporting test accuracy for all variants and picking the best.

- Inspecting only random errors instead of the most confident ones.

What to do next

Week 12: sequences: RNNs, LSTMs, embeddings, attention, mixed precision and profiling.

Weekly quiz: Week 11

1. Write the convolution output-size formula and compute it for n = 32, k = 5, p = 2, s = 2.
2. How many parameters does Conv2d(3, 16, 3) have, and how many would the equivalent dense layer need?
3. Give the timings for loops vs im2col vs torch from Day 71.
4. At which depth did the residual net beat the plain one on Day 72, and by how much?
5. Why did the MLP on CIFAR-10-small overfit more than the CNN?
6. What happened when augmentation was added on Day 73, in train and val accuracy?
7. Give the three transfer-learning results on Day 74.
8. What initial loss do you expect for 10 classes, and what did Day 75 measure?
9. What did the shuffled-label check show, and what would a failure have meant?
10. For fs 8,000 Hz and n_fft 256, give the bin width and number of bins.
11. Which four Fashion-MNIST pairs caused 58% of the Day-77 errors?
12. Define ECE and give the Day-77 value.

Answers

1. ?(n + 2p - k)/s? + 1 = 16.
2. 448; 44,251,200.
3. Roughly 208-244 ms, 2.1-3.1 ms, 0.27-0.39 ms across reruns (timings depend on machine load).
4. Depth 10: 0.7145 vs 0.6120, about +0.10.
5. No weight sharing or locality: 1.58M parameters each tied to a pixel position, so it memorises positions.
6. Train acc fell from 0.9372 to 0.7928, val acc from 0.7444 to 0.7268 in 10 epochs (smaller gap, no gain yet).
7. From scratch 0.3852, frozen features + LR 0.6424, fine-tune layer4+fc 0.6856.
8. ln 10 ~= 2.3026; measured 2.2930.
9. Test accuracy fell to 0.0889 (chance). A high value would mean leakage.
10. 31.25 Hz, 129 bins.
11. T-shirt/top-Shirt, Coat-Shirt, Pullover-Coat, Pullover-Shirt.
12. Confidence-binned average of |accuracy - confidence|, weighted by bin size; 0.0127.